Aviso inteligente de pesada cerrada

IA generativa para que la información de la balanza llegue a tiempo a quien tiene que decidir

Trabajo Final Integrador

Diplomatura en IA Aplicada a Entornos Digitales de Gestión

FCE-UBA · Cohorte 2026

Autor: Patricio *[COMPLETAR apellido y correo @campus.economicas.uba.ar]*

Repositorio: github.com/Patricio15623/tp-pesadas-ia

Simulador online: patricio15623.github.io/tp-pesadas-ia/simulador

Noviembre de 2026

a. Introducción

Elegí este tema porque trabajo hace más de diez años en administración del sector agropecuario y, en paralelo, desarrollé el sistema de pesadas que hoy usa una planta de alimento balanceado para registrar los camiones que entran con materia prima y salen con alimento. El registro ya está resuelto: cada pesada queda guardada con su bruto, tara, humedad y comprobante. Lo que todavía no está resuelto es que esa información **llegue a tiempo a quien tiene que decidir**. Si un camión trae humedad por encima del límite, si descargó mucho menos de lo habitual o si la tara no coincide con la del camión, o si entró un camión que nunca pasó por la balanza, hoy alguien tiene que estar mirando la pantalla para darse cuenta. Cuando se detecta en el reporte del día siguiente, el camión ya se fue y el reclamo es más difícil. *[COMPLETAR: agregar en una o dos oraciones una situación real (sin nombres) en la que esto haya pasado]*

La IA generativa aporta valor en un punto preciso: convertir números y alertas en un **mensaje que una persona entienda en cinco segundos desde el celular**, priorizando lo importante. Las cuentas, en cambio, no deben quedar en manos de un modelo de lenguaje, y buena parte del trabajo consistió en diseñar esa división de tareas.

Objetivo general

Diseñar y construir un prototipo funcional que, al cerrarse una pesada, envíe por Telegram un aviso redactado con IA, confiable en sus cifras y jerarquizado según la gravedad de lo detectado.

Objetivos específicos

- Detectar automáticamente situaciones que requieren atención: humedad sobre la base o sobre el límite, descuentos de calidad, neto y tara fuera de lo habitual, permanencia larga, datos incompletos.
- Cruzar cada pesada con las cámaras lectoras de patentes: patente que no coincide, lectura dudosa y camión que pasó por la cámara y no se pesó.
- Usar IA generativa solo para la redacción, con un control automático que impida enviar cifras inventadas.
- Garantizar que el aviso llegue aunque falle la IA o el servicio de mensajería.
- Permitir que cualquier persona evalúe el prototipo sin acceso al sistema real y sin datos reales, con una simulación fiel a la operación de la planta.

b. Marco conceptual

Modelos de lenguaje (LLM) y Claude. Un LLM genera texto a partir de instrucciones (prompt). Es muy bueno redactando y resumiendo, pero puede producir datos falsos con total seguridad (alucinaciones), especialmente con números. Para redactar los avisos se usó **Claude Haiku** (`claude-haiku-5-5`), el modelo chico y rápido de Anthropic, a través de su API; la estabilidad

del texto la dan el prompt de sistema y el validador (este modelo no admite ajustar la temperatura). Cada aviso cuesta fracciones de centavo de dólar. Como alternativa sin costo quedó preparado el mismo flujo con **Google Gemini** (`gemini-2.5-flash`, nivel gratuito).

Prompt de sistema y prompt de usuario. El prompt de sistema fija el rol y las reglas permanentes: qué puede y qué no puede hacer el modelo. El prompt de usuario lleva los datos de cada pesada. Separarlos permite mantener las reglas fijas y cambiar solo los datos.

n8n. Plataforma de automatización de flujos (low-code). Cada paso es un nodo: recibir un webhook, ejecutar código, llamar a una API o enviar un mensaje. Se usó la versión en la nube (n8n.cloud) para no depender de un servidor propio.

Vibe coding con Claude. Se usó Claude (Anthropic) como asistente de programación. Yo definía qué construir, con qué restricciones y cómo validarlo; la herramienta proponía y escribía código, que después se probaba y se corregía. **Telegram Bot API** es el canal de entrega, gratuito y ya instalado en los celulares del personal.

c. Metodología

Proceso seguido

1. **Delimitación.** Decidí no exponer el sistema real (código ni datos). Por eso todo el trabajo se apoya en una **simulación** con datos ficticios: proveedores inventados y patentes de una serie no emitida (ZZ), generados con un script de semilla fija.
2. **Diseño del evento.** Definí qué información viaja cuando se cierra una pesada: pesos, humedad, horarios, parámetros de calidad y un contexto histórico (neto promedio del proveedor y tara habitual del camión).
3. **Reglas antes que IA.** Escribí las reglas de negocio (merma = (H − base) / (100 − base), umbrales de desvío) en un único archivo con 33 pruebas automáticas.
4. **Flujo en n8n.** Webhook → reglas y control de acceso → Claude → validador → Telegram → respuesta.
5. **Simulador web** para que el docente pueda hacer una pesada completa (ingreso, tara, cierre) y ver el aviso, sin instalar nada. Decidí que reproduzca el aspecto y el flujo de las dos pantallas reales (la de la balanza y el tablero de la PC administrativa), para que la evaluación se vea como el trabajo cotidiano; la lectura del indicador de peso se reemplaza por una balanza simulada y todo lo propio del TP queda en un botón aparte.
6. **Pruebas de punta a punta** en un n8n local, reemplazando la IA y Telegram por un servidor simulado que responde bien, inventa cifras, omite la patente o falla.
7. **Fidelidad con la operación real (v4).** Revisé la simulación contra el sistema de la planta y la alineé en cinco puntos: mismo formato de operación, egreso con una carga por granja, descuentos de calidad, merma informativa por defecto con límite de recepción vacío = sin

aviso, y cámaras de patentes con las mismas reglas de unificación (5 minutos, repetidas en 20 segundos).

Cómo dirigí a la IA

Algunas de las instrucciones que le di a Claude durante el desarrollo, tal como las escribí:

- "¿Cómo ves para el trabajo final que me avise al bot de Telegram cuando una pesada fue cerrada?"
- "¿Lo podemos hacer para el repo simulando antes de ponerlo en práctica real?"
- "Andá trabajando en todo lo que puedas sin parar, respetando los requisitos." Los requisitos eran: datos ficticios, sin subir el sistema real, sin claves en el repositorio, y que se pueda probar sin mi PC encendida.
- "¿Lo estás llevando lo más cercano posible al sistema real?" La herramienta comparó la simulación con el sistema, detectó cinco diferencias y propuso corregirlas; decidí hacerlas todas y sumar la simulación de patentes.

Qué decidí yo: el tema y el canal; no publicar el sistema real; usar n8n.cloud; redactar con Claude usando mi propia API key (en lugar de Gemini, la primera opción propuesta); que la IA no haga cuentas; que el aviso sugiera pero nunca apruebe ni rechace un camión; proteger el webhook con una clave. **Qué resolvió la herramienta:** la estructura del simulador, el código de las reglas y del validador, el armado automático del flujo de n8n desde el mismo archivo de reglas, y las pruebas.

Evolución del prompt

Versión	Enfoque	Riesgo o problema
v1	"Resumí esta pesada para Telegram" + JSON crudo	La IA hace las cuentas y puede inventar cifras; formato variable.
v2	Fórmulas y umbrales dentro del prompt	Depende de la aritmética del modelo; cambiar un umbral es reescribir texto; no se puede probar.
v3 (vigente)	El código calcula; la IA recibe cifras ya escritas y solo redacta; un validador controla el texto	Ninguno conocido hasta ahora. Si la IA se equivoca, se envía la plantilla fija.

El prompt de sistema vigente tiene 7 reglas obligatorias. Entre ellas: usar solo los datos recibidos, copiar los kilos exactamente como vienen, incluir siempre la patente y el neto, ordenar las alertas por gravedad, cerrar con una sugerencia para una persona, un formato fijo por bloques con emojis (proveedor, producto, pesos, observaciones, sugerencia) y un máximo de 800 caracteres en texto plano. El texto completo está en `prompts/README.md`.

Qué salió mal y cómo lo corregí

- **Prueba del caso normal fallida:** la hora de salida de prueba era fija y generaba una alerta de "permanencia larga" que no correspondía. Se calculó la salida a partir de la entrada.

- **Salida anterior a la entrada:** la primera versión la interpretaba como “día siguiente” y mostraba 23 horas en planta. Se cambió por una alerta de horario incoherente, que es lo que realmente indica (un error de carga).
- **El prompt recibía “merma 0 kg”** aunque no hubiera merma, lo que invitaba a la IA a mencionarla. Se quitaron las cifras vacías.
- **Instalación de n8n para pruebas:** una dependencia se descargaba de un sitio bloqueado. Se resolvió forzando una versión equivalente del registro oficial.
- **La salida de otro camión se tomaba como patente distinta:** al cerrar un ingreso, la cámara de salida había leído otro camión en el mismo horario y el aviso lo informaba como discrepancia. Se corrigió considerando solo los pasos de ENTRADA y se agregó una prueba.
- **Detalles de redacción** en la plantilla (“INGRESO CERRADA”), corregidos al revisar la salida.

d. Resultados

El repositorio contiene un prototipo funcional (nivel 1), probado de punta a punta: simulador web publicado, flujo de n8n importable, script de terminal, datos ficticios, 33 pruebas automáticas y documentación. Repositorio: https://github.com/Patricio15623/tp-pesadas-ia. Simulador: https://patricio15623.github.io/tp-pesadas-ia/simulador/. Canal público con los avisos reales, visible sin cuenta de Telegram: https://t.me/s/balanza_avisos_tp.

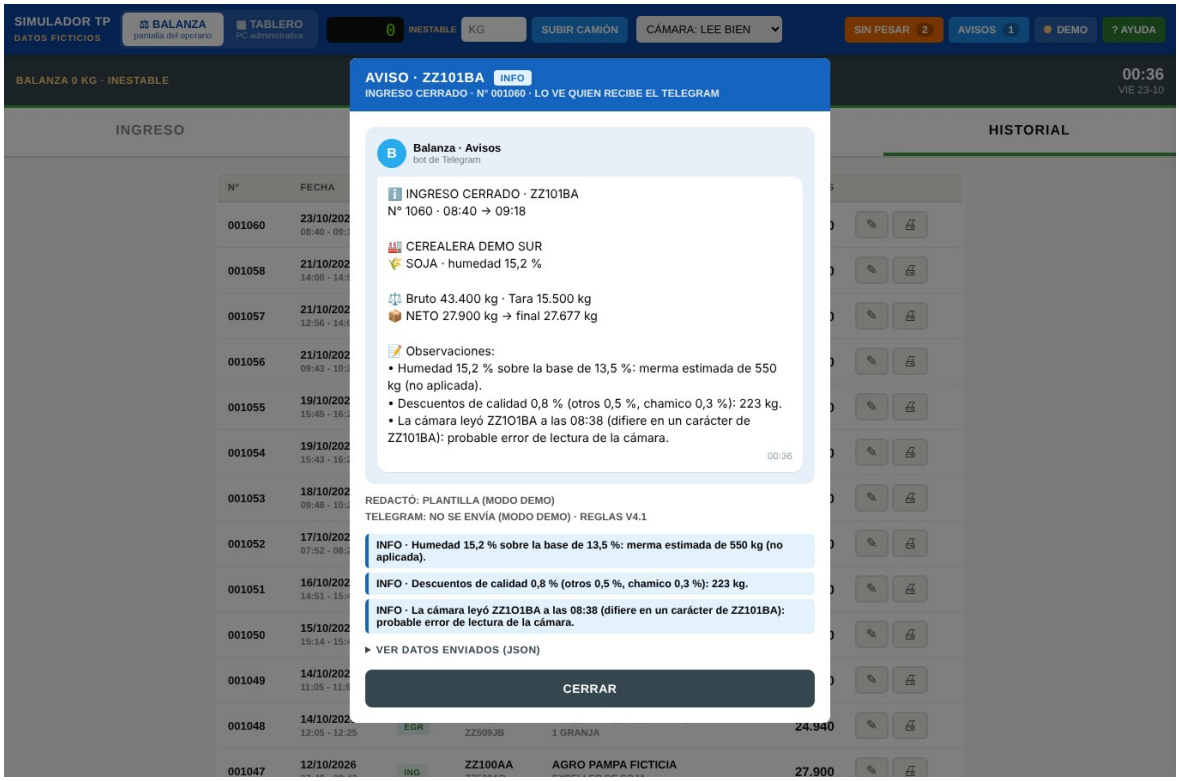


Figura 1. Simulador con el aspecto de la pantalla real de la balanza: al cerrar una pesada aparece el aviso tal como llegaría a Telegram, con las alertas detectadas.

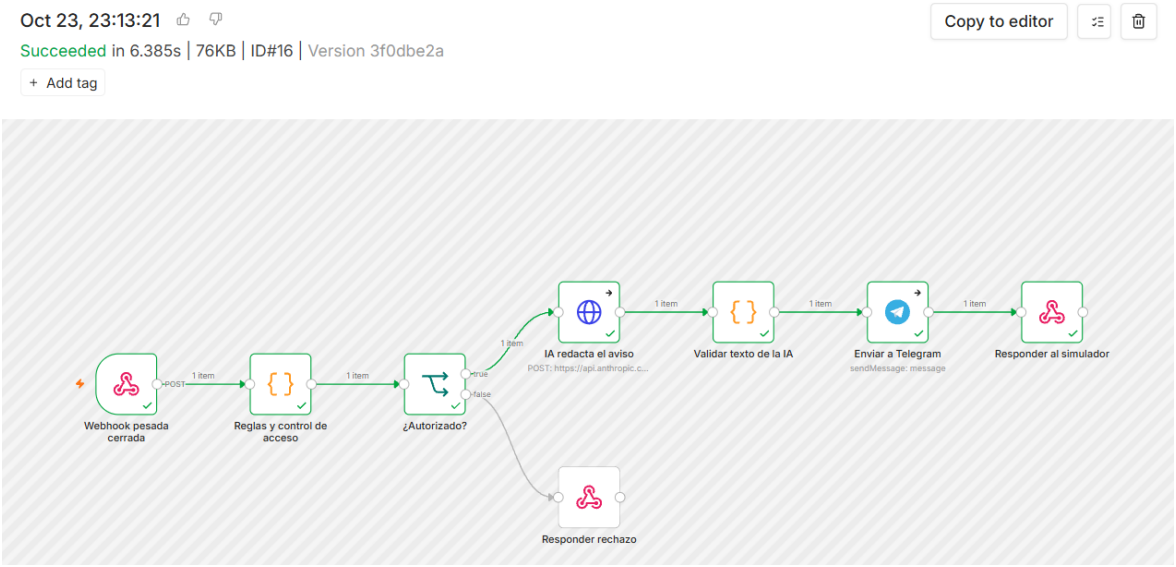


Figura 2. Flujo en n8n.cloud: ejecución real completa (IA, validador y Telegram). La rama inferior responde 401 si la clave no coincide.

Escenario (datos ficticios)	Nivel	Qué detecta el aviso
Ingreso normal de maíz	✅ NORMAL	Neto 29.600 kg, sin observaciones
Soja con humedad 15,2 % y descuentos	ℹ️ INFO	Merma estimada 550 kg (no aplicada), descuentos 0,8 %, lectura dudosa de la cámara (0 por O)
Maíz con humedad 18,6 %	🛑 CRÍTICO	Supera el límite de recepción cargado por la planta (18 %)
Expeller con poco neto	⚠️ ATENCIÓN	7.375 kg bajo el promedio, tara 875 kg mayor y la cámara leyó otra patente
Egreso a tres granjas	✅ NORMAL	Neto por granja: 9.200 + 10.400 + 8.100 kg
Egreso con 3 h 20 min en planta	⚠️ ATENCIÓN	Permanencia larga; vehículo sin patente legible
Camión ZZ110KA	⚠️ ATENCIÓN	Pasó por la cámara y no tiene pesada

Prueba de robustez del flujo	Resultado
La IA agrega una cifra no calculada (“3.210 kg”)	Texto descartado; se envía la plantilla
La IA omite la patente	Texto descartado; se envía la plantilla
La IA no responde (error 500)	Se envía la plantilla con el motivo
Telegram no responde	El aviso se genera y se informa “no enviado”
Clave incorrecta	Rechazo 401, sin consumir la IA

Prueba con servicios reales (22 y 23/10/2026). Con n8n.cloud, Claude Haiku y un canal de Telegram, los avisos llegaron redactados por la IA con el formato pedido y las cifras exactas. En el caso de ATENCIÓN, el modelo ordenó las alertas por importancia y resumió en una sola sugerencia las tres verificaciones (mismo camión, acoplado y carga remanente). La puesta en marcha mostró cuatro problemas que la prueba local no podía detectar: el nombre del encabezado de la credencial, un parámetro (temperature) que este modelo ya no acepta, un identificador de chat mal copiado y un aviso que llegó cortado porque el límite de tokens era bajo. Este último pasó el validador, así que se subió el límite y se agregaron dos controles: se descarta

el texto si la respuesta terminó por límite de tokens o si a un aviso de ATENCIÓN o CRÍTICO le falta la sugerencia final.

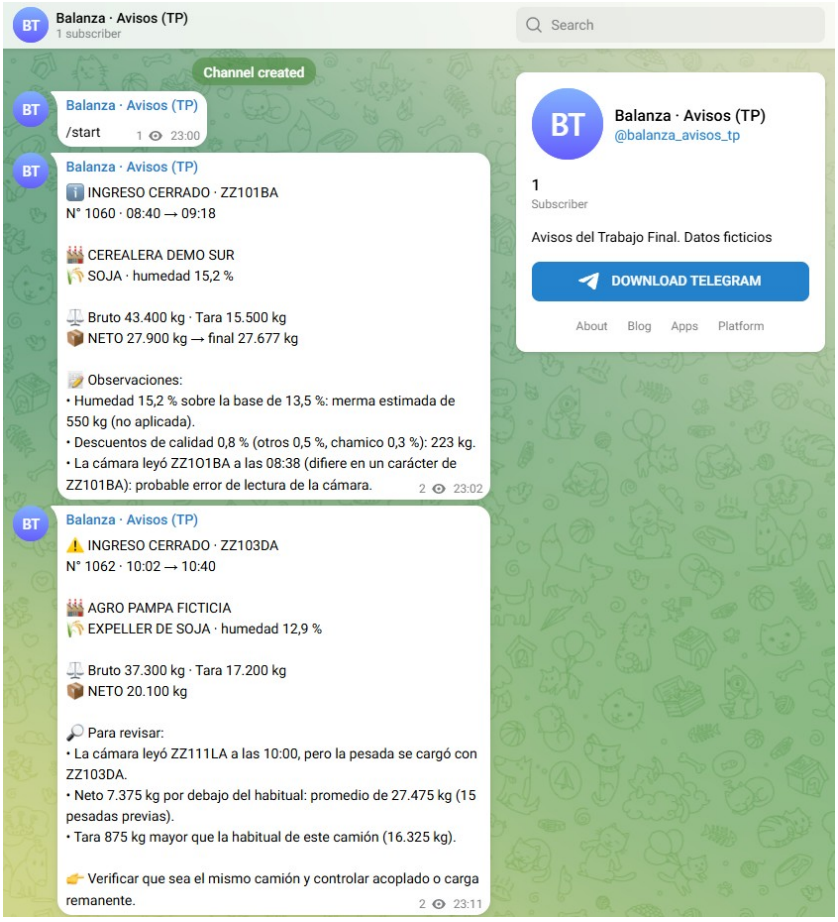


Figura 3. Avisos reales redactados por Claude en el canal público de Telegram (INFO y ATENCIÓN).

e. Análisis crítico

Fortalezas

- **Cifras confiables:** la IA nunca calcula. Un validador descarta cualquier número que el sistema no haya calculado.
- **El aviso llega siempre:** si fallan la IA o Telegram, hay respaldo y queda registrado el motivo.
- **Una sola fuente de reglas:** el simulador y n8n usan el mismo archivo, con pruebas automáticas.
- **Evaluable sin depender de nadie:** el modo demo funciona sin conexión y con datos ficticios.

Limitaciones

- Depende de servicios en la nube: la prueba gratuita de n8n.cloud y el saldo de la API de Anthropic. Si alguno se corta, el modo conectado deja de funcionar, aunque la demo sigue andando y el flujo alternativo con Gemini no tiene costo.

- El historial y las lecturas de cámaras son simulados. En la planta, el contexto (promedios, tara habitual, lecturas cercanas) debería aportarlo el sistema real, que ya registra las cámaras.
- El cruce con la cámara usa una ventana fija alrededor de la entrada (15 min antes, 10 después); en horas pico podría requerir ajuste.
- La clave del webhook viaja en el link que se comparte: protege contra el uso casual, no contra alguien que la conozca.
- Los umbrales (15 % de desvío de neto, 3 % de tara, 2 h de permanencia) son de ejemplo y deberían definirlos los responsables de la planta.
- *[COMPLETAR: limitaciones observadas en la redacción real de Claude, si las hubo]*

Oportunidades

- Conectar el sistema real: enviar el mismo JSON al cerrar cada pesada, sin cambiar el flujo.
- Resumen diario o semanal redactado por IA: kilos por proveedor, mermas, rechazos.
- Consultas en lenguaje natural al bot (“¿cuánto maíz entró esta semana?”) con respuestas calculadas por el sistema.
- Asociar automáticamente la lectura de la cámara a la pesada al momento de pesar, para que el operario no tipee la patente.

Evaluación AIBPS

Dimensión	Evaluación
Ágil	Se implementa con herramientas existentes (n8n, Telegram) sin servidor propio. Cambiar un umbral o el prompt es editar un archivo y regenerar el flujo.
Fluida	No agrega pasos al operario: el aviso sale solo al cerrar la pesada, usa los mismos datos que el sistema ya registra (incluidas las cámaras) y llega al celular en lenguaje claro, ordenado por gravedad.
Protegida	Datos ficticios en el repositorio; credenciales (API key, token del bot) fuera del código; webhook con clave; validación de cifras; respaldo si falla la IA. Con datos reales habría que revisar las condiciones de uso de datos del proveedor de IA.
Bajo control humano	La IA solo redacta. Las reglas son explícitas y auditables, y el aviso sugiere revisar pero nunca aprueba ni rechaza: decide una persona.

f. Conclusiones

El aprendizaje principal fue **decidir qué no hacer con IA**. En un entorno donde un número equivocado se convierte en un reclamo o en una liquidación mal hecha, la IA generativa rinde cuando se la usa para lo que hace bien (redactar, priorizar, explicar) y se la rodea de controles para lo que hace mal (calcular, ser exacta). El patrón “el código calcula, la IA redacta, el código valida” se puede trasladar a otros procesos de gestión: conciliaciones, reportes de stock, alertas de cobranzas.

También aprendí que dirigir a una IA de programación se parece a coordinar a un colaborador muy rápido. Funciona cuando las restricciones están claras desde el principio (qué no se publica, qué datos usar, cómo se prueba) y cuando cada resultado se verifica, en vez de darlo por bueno.

Recomendaciones: probar el aviso con usuarios reales durante dos semanas antes de ajustar los umbrales; medir cuántos avisos de ATENCIÓN terminan en una acción concreta, para evitar que se conviertan en ruido; y, con datos reales, pasar a un proveedor y un plan de IA cuyas condiciones garanticen que la información no se use para entrenamiento. *[COMPLETAR: una reflexión personal de cierre]*